



Чему научиться в этом году?

[Войти](#)**LanskoyGames**

10 мар 2024 в 11:17

Создание генетического алгоритма для нейросети и нейросети для графических игр и видеоигр с помощью Python и NumPy

Средний 6 мин 8.8K

Python*, Программирование*, Алгоритмы*, Машинное обучение*, Искусственный интеллект

[Тutorial](#)

Привет, Хабр!

Сегодня я расскажу и покажу, как сделать Genetic Algorithm(GA) для нейросети, чтобы с помощью него она смогла проходить разные игры. Я его испробовал на игре Pong и Flappy bird. Он себя показал очень хорошо. Советую прочитать, если вы не читали первую статью: «Создание простого и работоспособного генетического алгоритма для нейросети с Python и NumPy», так как я доработал свой код, который был показан в той статье.

Я разделил код на два скрипта, в одной нейросеть играет в какую-то игру, в другой обучается и принимает решения(сам генетический алгоритм). Код с игрой представляет из себя функцию которая возвращает фитнес функцию (она нужна для сортировки нейросетей, например, сколько времени она продержалась, сколько очков заработала и т.п.). Поэтому код с играми(их две) будет в конце статьи. Генетический алгоритм для нейросети для игры Pong и игры Flappy Bird различаются лишь параметрами.

Используя скрипт, который я написал и описал в предыдущей статье, я создал сильно изменённый код генетического алгоритма для игры Pong, который я и буду описывать больше всего, так как именно на него я опирался, когда я уже создавал GA для Flappy Bird.

Вначале нам потребуется импортировать модули, списки и переменные:

```

import numpy as np
import random
import ANNPong as anp
import pygame as pg
import sys
from pygame.locals import *
pg.init()
listNet = {}
NewNet = []
goodNet = []
timeNN = 0
moveRight = False
moveLeft = False
epoch = 0
mainClock = pg.time.Clock()
WINDOWWIDTH = 800
WINDOWHEIGHT = 500
windowSurface = pg.display.set_mode((WINDOWWIDTH, WINDOWHEIGHT), 0, 32)
pg.display.set_caption('ANN Pong')

```

Объяснить с 

AnnPong это скрипт с игрой

listNet, NewNet, goodNet - списки нейросетей(потом разберем подробнее)

timeNN - фитнес функция

MoveRight, moveLeft - выбор нейросети куда двигаться

epoch - счетчик эпох

```

def sigmoid(x):
    return 1/(1 + np.exp(-x))

class Network():
    def __init__(self):
        self.H1 = np.random.randn(6, 12)
        self.H2 = np.random.randn(12, 6)
        self.O1 = np.random.randn(6, 3)
        self.BH1 = np.random.randn(12)
        self.BH2 = np.random.randn(6)
        self.BO1 = np.random.randn(3)
        self.epoch = 0

```

```

def predict(self, x, first, second):
    nas = x @ self.H1 + self.BH1
    nas = sigmoid(nas)
    nas = nas @ self.H2 + self.BH2
    nas = sigmoid(nas)
    nas = nas @ self.O1 + self.B01
    nas = sigmoid(nas)
    if nas[0] > nas[1] and nas[0] > nas[2]:
        first = True
        second = False
        return first, second
    elif nas[1] > nas[0] and nas[1] > nas[2]:
        first = False
        second = True
        return first, second
    elif nas[2] > nas[0] and nas[2] > nas[1]:
        first = False
        second = False
        return first, second
    else:
        first = False
        second = False
        return first, second
def epoch(self, a):
    return 0

```

```

class Network1():
    def __init__(self, H1, H2, O1, BH1, BH2, B01, ep):
        self.H1 = H1
        self.H2 = H2
        self.O1 = O1
        self.BH1 = BH1
        self.BH2 = BH2
        self.B01 = B01
        self.epoch = ep

    def predict(self, x, first, second):
        nas = x @ self.H1 + self.BH1
        nas = sigmoid(nas)
        nas = nas @ self.H2 + self.BH2
        nas = sigmoid(nas)
        nas = nas @ self.O1 + self.B01
        nas = sigmoid(nas)
        if nas[0] > nas[1] and nas[0] > nas[2]:

```

```

        first = True
        second = False
        return first, second
    elif nas[1] > nas[0] and nas[1] > nas[2]:
        first = False
        second = True
        return first, second
    elif nas[2] > nas[0] and nas[2] > nas[1]:
        first = False
        second = False
        return first, second
    else:
        first = False
        second = False
        return first, second

```

Объяснить с 

Сигмоида используется как функция активации.

В классе Network мы определяем параметры нейросети, а в функции predict она говорит, куда двигаться в игре. (nas - сокращение от Network answer), функция epoch возвращает эпоху появления этого ИИ для нулевого поколения, так как в классе Network1() для этого задается отдельная переменная.

```

for s in range (1000):
    s = Network()
    timeNN = anp.NNPong(s)
    listNet.update({
        s : timeNN
    })

listNet = dict(sorted(listNet.items(), key=lambda item: item[1]))
NewNet = listNet.keys()
goodNet = list(NewNet)
NewNet = goodNet[:10]
listNet = {}
goodNet = NewNet
anp.NPong(NewNet[0])
print(str(epoch) + " epoch")
print(NewNet[0].epoch)
print('next')
anp.NPong(NewNet[1])
print(NewNet[1].epoch)
print('next')

```

```

anp.NPong(NewNet[2])
print(NewNet[2].epoch)
print('next')
anp.NPong(NewNet[3])
print(NewNet[3].epoch)
print('next')
anp.NPong(NewNet[4])
print(NewNet[4].epoch)
print('next')
anp.NPong(NewNet[5])
print(NewNet[5].epoch)
print('next')
anp.NPong(NewNet[6])
print(NewNet[6].epoch)
print('next')
anp.NPong(NewNet[7])
print(NewNet[7].epoch)
print('next')
anp.NPong(NewNet[8])
print(NewNet[8].epoch)
print('next')
anp.NPong(NewNet[9])
print(NewNet[9].epoch)
print('that is all')

```

Объяснить с 

Здесь мы прогоняем нейросети со случайно созданными весами и выбираем из них 10 самых худших, чтобы всю работу по их воспитанию брал на себя генетический алгоритм))) и показываем их.

Подробнее:

В timeNN записывается возвращенная из кода с игрой фитнес функция, затем мы добавляем в listNet ИИ и его значение timeNN. После цикла мы сортируем список, записываем в NewNet нейросети из listNet, дальше мы формируем список и оставляем только десять.

```

for g in range(990):
    parent1 = random.choice(NewNet)
    parent2 = random.choice(NewNet)
    ch1H = np.vstack((parent1.H1[:3], parent2.H1[3:])) * random.uniform(-2, 2)
    ch2H = np.vstack((parent1.H2[:6], parent2.H2[6:])) * random.uniform(-2, 2)
    ch10 = np.vstack((parent1.O1[:3], parent2.O1[3:])) * random.uniform(-2, 2)
    chB1 = parent1.BH1 * random.uniform(-2, 2)

```

```

chB2 = parent2.BH2 * random.uniform(-2, 2)
chB3 = parent2.B01 * random.uniform(-2, 2)
g = Network1(ch1H, ch2H, ch10, chB1, chB2, chB3, 1)
goodNet.append(g)
NewNet = []

```

Объяснить с 

Здесь происходит скрещивание и мутация.(Такие моменты более подробно были описаны в первой статье)

```

while True:
    epoch += 1
    print(str(epoch) + " epoch")
    for s in goodNet:
        timeNN = anp.NNPong(s)
        listNet.update({
            s : timeNN
        })
    goodNet =[]
    listNet = dict(sorted(listNet.items(), key=lambda item: item[1], reverse=True))
    goodNet = list(listNet.keys())
    NewNet.append(goodNet[0])
    goodNet = list(listNet.values())
    for i in listNet:
        a = goodNet[0]
        if listNet.get(i) == a:
            NewNet.append(i)
    goodNet = list(NewNet)
    listNet = {}
    try:
        print(NewNet[0].epoch)
        anp.NPong(NewNet[0])
        print('next')
        print(NewNet[1].epoch)
        anp.NPong(NewNet[1])
        print('next')
        print(NewNet[2].epoch)
        anp.NPong(NewNet[2])
        print('next')
        print(NewNet[3].epoch)
        anp.NPong(NewNet[3])
        print('next')
        print(NewNet[4].epoch)
        anp.NPong(NewNet[4])

```

```

print('next')

print(NewNet[5].epoch)
anp.NPong(NewNet[5])
print('next')
print(NewNet[6].epoch)
anp.NPong(NewNet[6])
print('next')
print(NewNet[7].epoch)
anp.NPong(NewNet[7])
print('next')
except IndexError:
    print('that is all')

for g in range(1000 - len(NewNet)):
    parent1 = random.choice(NewNet)
    parent2 = random.choice(NewNet)
    ch1H = np.vstack((parent1.H1[:3], parent2.H1[3:])) * random.uniform(-2,
    ch2H = np.vstack((parent1.H2[:6], parent2.H2[6:])) * random.uniform(-2,
    ch10 = np.vstack((parent1.O1[:3], parent2.O1[3:])) * random.uniform(-
    chB1 = parent1.BH1 * random.uniform(-2, 2)
    chB2 = parent2.BH2 * random.uniform(-2, 2)
    chB3 = parent2.B01 * random.uniform(-2, 2)
    g = Network1(ch1H, ch2H, ch10, chB1, chB2, chB3, epoch)
    goodNet.append(g)
print(len(NewNet))
print(len(goodNet))
NewNet = []

```

Объяснить с 

Здесь уже пошло повторение, поэтому объясню только то, что не было сказано до этого:

Здесь мы берём первого в списке, то есть одного из лучших в эпохе и сверяем его результаты с остальными, так как очень часто есть несколько ИИ, которые добились таких же успехов. И эти равноправные лидеры будут участвовать в мутациях, мы используем метод `try`, так как лучших в этой эпохе может быть меньше 10. А ещё мы закидываем эти нейросети в следующую эпоху без изменений, так как потомки могут оказаться хуже их предков, то есть чтобы они не деградировали.

Это всё по первому коду!

Перейдем к коду игры. Тут я объясню только то, что касается обучения ИИ(весь размещу ссылкой на диск).

В игре Pong нейросеть играла дважды: в первый раз мячик отскакивает влево, второй раз - вправо

*whGo - это переменная в коде(сокращение от "where to go")

Мы возвращаем время, как фитнес функцию. В игре есть две почти одинаковые функции, но во второй мы показываем все на экране, это нужно, чтобы мы видели прогресс после каждой эпохи и когда нейросеть прошла игру, мы это определяем, если она продержалась в первой больше 8000 тысяч обновлений.

После месяцев работы и доработок, у меня получилось создать алгоритм обучения для игры Pong, однако для уверенности я решил проверить ИИ не на своей игре, а созданную другим человеком(проверка на всеядность))), я выбрал игру Flappy Bird на ругаме с этого видео: <https://youtu.be/7lqrZb0Sotw?feature=shared>

Немного изменив игру для нейросети, например, добавил переменные расстояния от птицы до трубы. Их 3 по 3, так как нам нужно знать высоту каждой трубы(y) и расстояние по x, а на экране не было больше трех пар труб, поэтому и три по три(всего девять). Также после столкновения функция перезапускалась и третьим параметром, который назван per функции передавалось какой это перезапуск, если он был равен трем, то игра возвращала фитнес функцию в Genetic Algorithm, а если нулю, то мы присуждаем переменной time значение 0. Также я не писал две очень похожие друг на друга функции, а просто проверял, если переменная checkNN равна True, то нужно обновить экран.

Я также доработал код обучения

```
while True:
    for event in pg.event.get():
        if event.type == KEYDOWN:
            if event.key == K_1:
                showNN = True
    epoch += 1
    print(str(epoch) + " epoch")
    if epoch < 10:
        for s in goodNet:
            timeNN = anp.NPong(s, False, 0, 0)
            listNet.update({
                s : timeNN
            })
    if epoch >= 10:
        for s in goodNet:
            timeNN = anp.NPong(s, False, 0, 1)
            listNet.update({
```



```
s : timeNN  
})
```

Объяснить с 

После десятой эпохи из-за последнего параметра, который мы меняем на единицу(в коде игры я назвал этот параметр varRe от слов variant of return), игра возвращает не время, а кол-во труб до столкновения(так нейросеть учиться лучше)

```
howALot = 1000 - len(NewNet)  
if howALot < 40:  
    howALot = 40
```

Объяснить с 

Эти три строки кода нужны, если в предыдущей эпохе ИИ с одинаковым результатом оказалось очень, очень много и алгоритм может перестать обучаться, так как ему будет нечего обучать :-).

My neural network is playing Pong

LanskoyGames

My neural network is playing Flappy Bird

LanskoyGames

На этом всё, если есть вопросы, пишите в комментариях, пока!

Эта же моя статья, но на DTF или dtf: <https://dtf.ru/u/1361840-kirill-lanskoi/2888076-sozдание-geneticheskogo-algoritma-dlya-neiroseti-i-neiroceti-dlya-graficheskikh-igr-s-pomoshyu-python-i-numpy>.

Коды: <https://github.com/LanskoyKirill/GenNumPy.git>

Теги: генетический алгоритм, ии, нейросети, python, numpy, обучение, neat, искусственный интеллект, genetic algorithm, игр

Хабы: Python, Программирование, Алгоритмы, Машинное обучение, Искусственный интеллект

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



17

Карма

@LanskoyGames

Пользователь

Подписаться



 Комментарии 4

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



Guren302

23 часа назад

Как ИИ убивает игровую индустрию (и не только её)



Простой



10 мин



14K

Обзор



+35



20



59



radiolok

4 часа назад

Стыдно когда видно! Советское видеонаблюдение



Простой



12 мин



4.4K

Обзор



+28



2



9



TrexSelectel

4 часа назад

Podman Compose: как устроен, что умеет, чем отличается от Docker Compose



5 мин



3.3K



+25



8



1



gtosss
9 часов назад

Найм в IT, 2026: 2 вакансии, 1 000 откликов в сутки



Простой



11 мин



11K

Мнение



+25



14



72



linux-over
18 часов назад

Строим VPN, устойчивый к SpyWare



Сложный



6 мин



21K

Аналитика



+25



108



89



Sonia_Black
18 часов назад

Мой CLAUDE.md — 582 строки. Вот зачем



Средний



6 мин



22K

Кейс



+25



150



34



RationalAnswer
7 часов назад

Приземление нутелловых космонавтов, а также деанон Сатоши Накамото



9 мин



15K

Дайджест



+23



5



15



morett1m
5 часов назад

Как Rust реализует трейт-объекты и почему dyn Trait медленнее дженериков



11 мин



3.9K

Обзор

+20

5

0



Catx2

22 часа назад

Трубы горят... и летят

24 мин

12K

+20

15

3



eliza_petyaeva

3 часа назад

Динамическое ценообразование в E-grocery. Раскладываем по полочкам

Средний

11 мин

3.1K

Обзор

+18

12

1

От реле до нейросети: лучшие статьи сезона Heavy Digital

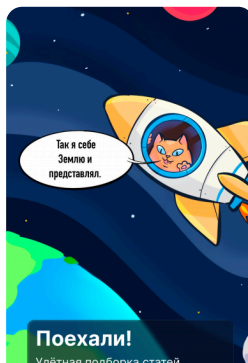
Турбо

Показать еще

ИСТОРИИ



A



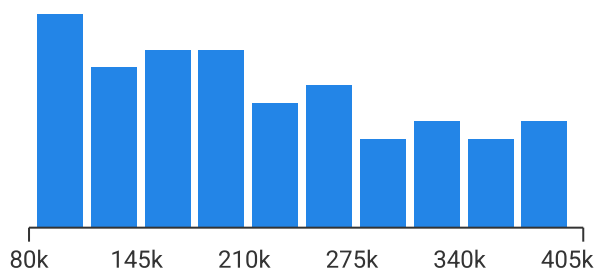
Хочу 450K

Вперёд к звёздам

СРЕДНЯЯ ЗАРПЛАТА В IT

222 390 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 30 571 анкеты, за 1-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!

[Проверить свою зарплату](#)

МИНУТОЧКУ ВНИМАНИЯ



Многорукий 1С: как мы проапгрейдили его для фармкомпаний



Отношение айтишников к ИИ: результаты исследования



Про IT на заводах: лучшие статьи сезона Heavy Digital

БЛИЖАЙШИЕ СОБЫТИЯ



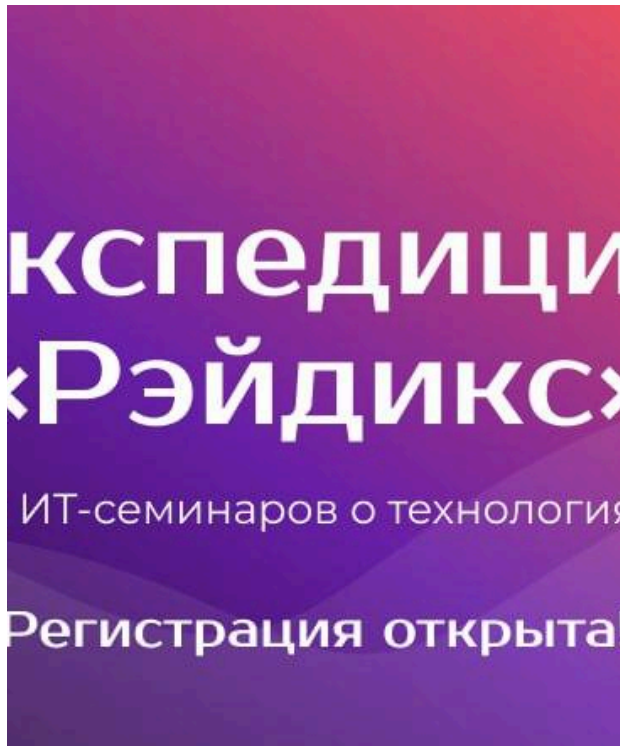
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная премия,
учрежденная в 2000 году**

Санкт-Петербург

[Маркетинг](#)

[Больше событий в календаре](#)



3 марта – 9 июня

Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск • Краснодар •
Минск • Калининград • Ростов-на-Дону

[Разработка](#) [Другое](#)

[Больше событий в календаре](#)



9 марта

**Всё,
стар
брен**

Онлайн

[Марк](#)

[Больше](#)



 [Настройка языка](#)

[Техническая поддержка](#)

